

Deep Learning for Solving Dynamic Models

Wrap-up: what to use when, and what to watch out for

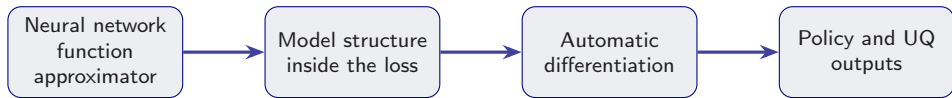
Simon Scheidegger

HEC, University of Lausanne

University of Padova

September 24, 2026

The Big Picture: One Computational Paradigm



Across methods

DEQN, PINN, deep surrogates, and GP-BAL differ in objective and geometry, but share the same core optimization logic.

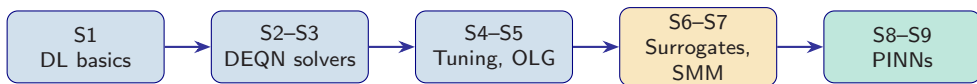
Course Arc: Two Days

Session	Step in the pipeline
1	Deep learning foundations, optimization, and generalization
2	DEQNs: equilibrium residuals as an unsupervised loss (Brock–Mirman)
3	Scaling to a $2N$ -dimensional state space (IRBC)
4	Making training work: architecture search and loss balancing
5	Cohort structure and constraints (OLG, 56 generations)
6	Amortizing the solve: deep surrogates and Gaussian processes
7	Structural estimation via SMM on top of a surrogate
8–9	Continuous time: PINNs for ODEs, PDEs, HJB and option pricing

Main message

The toolkit is modular: solve the model, amortize the solve, then estimate, each layer reuses the one below it.

Visual Timeline of the Course Workflow



Core paradigm:
model structure in the loss + autodiff + scalable function approximation

Decision Guide: Which Method When?

Method	Use case	Strength
DEQN	Recursive equilibrium with explicit Euler residuals	Scales to high-dimensional states
PINN	Continuous-time PDE / HJB / KFE systems	Mesh-free; no tensor grid to refine
Finite difference	Low-dimensional continuous-time problems	Fast, accurate, well understood
Deep surrogate	Expensive simulator queried many times	Amortizes the solve; differentiable
GP + BAL	Smooth low/medium-dimensional maps	Built-in UQ and sample efficiency

Rule of thumb

Reach for a neural method when the state space defeats a tensor grid, not before. In two or three dimensions the classical solver usually wins.

Common Failure Modes and Mitigations

Failure modes

- ▶ Calibration mismatch to benchmark science
- ▶ Training focused on wrong state regions
- ▶ Over-reliance on mean loss diagnostics
- ▶ Surrogate extrapolation outside training support

Mitigations

- ▶ Explicit benchmark tests and stress scenarios
- ▶ Mixed exogenous/endogenous sampling
- ▶ Report residual distributions and constraints
- ▶ Active learning and uncertainty-aware evaluation

Implementation Checklist

1. Define state/control vectors and feasibility transforms.
2. Engineer residual blocks from economics and climate equations.
3. Set sampling scheme and training diagnostics before tuning architecture.
4. Validate on out-of-sample simulations and benchmark moments.
5. Build surrogate/UQ layer only after solver quality is verified.
6. Separate normative objective from political/participation constraints.

What We Did Not Cover

Each of these is a natural next step from where we stopped:

- ▶ **Sequence-space DEQNs:** transition paths and MIT shocks.
- ▶ **Krusell–Smith and Young’s method:** a continuum of agents with a distribution as a state variable.
- ▶ **Climate economics and IAMs:** DICE with DEQNs, deep uncertainty quantification, constrained Pareto-improving carbon taxes.
- ▶ **Continuous-time heterogeneous agents:** the coupled HJB–KFE system, continuous-time Aiyagari, and the master equation.
- ▶ **Agentic programming:** AI coding agents as research partners.

- ▶ Richer heterogeneity: regional, sectoral, and household dimensions.
- ▶ Estimation at scale: likelihood-based and simulation-based inference on top of amortized solvers.
- ▶ Better theoretical guarantees: convergence, error bounds, and verification of trained policies.
- ▶ Operator learning: mapping whole parameterizations to solutions, rather than one model at a time.
- ▶ Robustness: model ensembles and policy design under ambiguity.

Closing Message

From method to model

These methods earn their place when they make a model **tractable** that was not, without giving up the **diagnostics** that let you trust the answer. Always keep a benchmark you can check against.

Thank you for participating in the course.

Questions and discussion